

# IAM-04: Policy Authoring and Management

---

**Series:** IAM — IAM Administration | **Notebook:** 4 of 12 | **Created:** January 2026 |  
**Last Updated:** 09/24/2026

## Mastering Dynatrace Policy Syntax

---

Policies are the heart of Dynatrace Gen3 IAM. They define what actions users can perform. This notebook provides a comprehensive guide to policy authoring, from basic syntax to advanced patterns.

---

## Table of Contents

---

1. [Policy Fundamentals](#)
  2. [Policy Statement Syntax](#)
  3. [Services and Actions Reference](#)
  4. [Condition Expressions](#)
  5. [Common Policy Patterns](#)
  6. [Default vs Custom Policies](#)
  7. [Policy Testing and Validation](#)
  8. [GitOps for Policies](#)
  9. [Policy Assessment Queries](#)
-

# Prerequisites

| Requirement           | Details                                                        |
|-----------------------|----------------------------------------------------------------|
| Dynatrace Environment | SaaS with Gen3 IAM enabled                                     |
| Permissions           | <code>account-user-management</code> to create/modify policies |
| Prior Knowledge       | <b>IAM-01</b> through <b>IAM-03</b>                            |

## SaaS 1.337: `frontend.name` as a permission field

SaaS 1.337 added `frontend.name` as a condition for metrics, RUM and Smartscape data, so access can be scoped per frontend without splitting buckets. The condition is `storage:frontend.name` on each table permission, and the policy reference lists `=`, `IN`, `startsWith` and `MATCH` for it:

```
ALLOW storage:metrics:read WHERE storage:frontend.name IN ("checkout-web",
"account-web");
ALLOW storage:user.events:read WHERE storage:frontend.name IN ("checkout-web",
"account-web");
ALLOW storage:user.sessions:read WHERE storage:frontend.name MATCH ("checkout-
web", "account-web");
```

Use `MATCH` for `user.sessions`: a session can span several frontends. Check each value in Experience Vitals before binding the policy, because the `frontend.name` value is not always the display name.

### Sources:

- [SaaS 1.337 \(DT docs\)](#) — "Statements like `ALLOW storage:metrics:read WHERE storage:frontend.name = "mobile_app"` let you selectively control access to metrics and Smartscape data at the record- and field level."
- [RUM data access controls best practices \(DT docs\)](#) — "Because user sessions may span multiple frontends, use the `MATCH` operator"; "The `frontend.name` value may differ from the display name shown in Experience Vitals."
- [IAM policy reference \(DT docs\)](#)

# 1. Policy Fundamentals

Policies define **what actions** users can perform. They're separate from boundaries (which control **what data** users can see).

**One policy language for all authorization.** Dynatrace's security policies are the single configuration surface for every authorization requirement — environment access, Grail data access, and AppEngine capabilities alike. What used to be split between classic *roles* (RBAC — "viewer", "admin") and attribute-based permissions is now unified: classic roles are expressed as `environment:roles:*` permissions inside the same `ALLOW` grammar as `storage:*` and `app-engine:*`. There is no separate RBAC subsystem to manage — one policy, one language, optionally scoped by `WHERE` conditions or boundaries.

## Policy Scope Levels

| Scope       | Applies To                  | Managed By        |
|-------------|-----------------------------|-------------------|
| Account     | All environments in account | Account Admin     |
| Environment | Single environment          | Environment Admin |

## Policy Assignment

Policies are assigned to **groups**, not individual users:

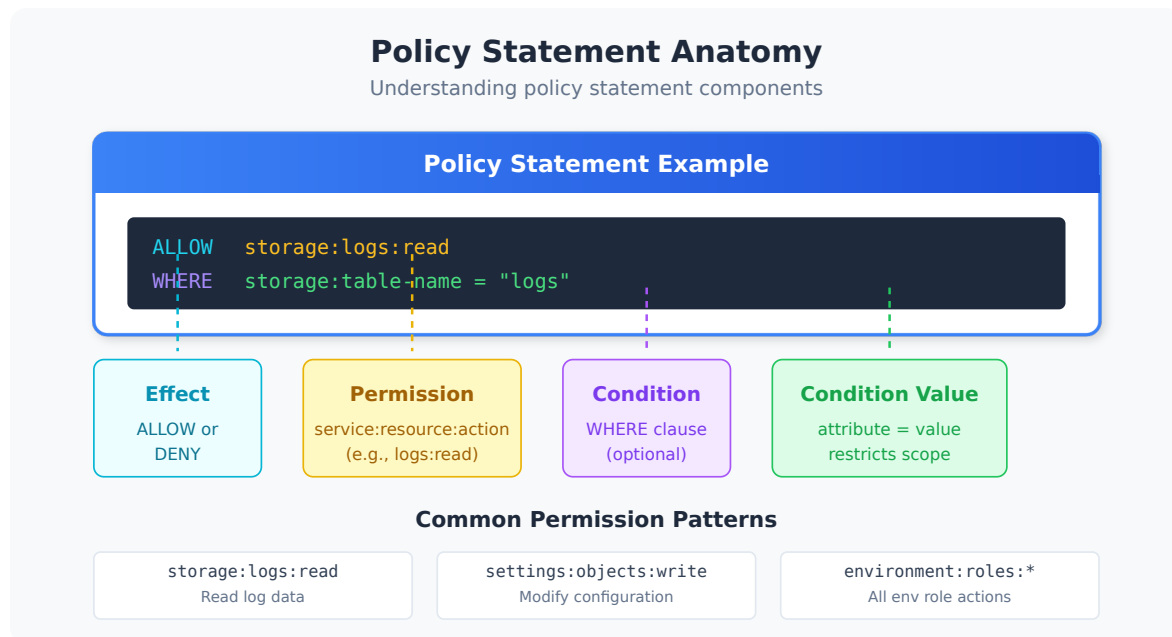
```
Group: dt-checkout-editors
├─ Policy: Pro User (default)
├─ Policy: log-management (custom)
└─ Boundary: checkout-services-only
```

## Policy Types

| Type             | Description            | Examples                                                    |
|------------------|------------------------|-------------------------------------------------------------|
| Default Policies | Pre-built by Dynatrace | Standard User, Pro User, Admin User                         |
| Custom Policies  | Created by you         | <code>log-read-only</code> , <code>dashboard-manager</code> |

## 2. Policy Statement Syntax

Every policy contains one or more **statements** that define allowed actions.



### Basic Statement Structure

```
ALLOW <service>:<resource>:<action>;
```

Examples:

| Statement                     | Meaning                   |
|-------------------------------|---------------------------|
| ALLOW storage:logs:read       | Read all logs             |
| ALLOW storage:spans:read      | Read all spans            |
| ALLOW settings:objects:write  | Modify settings           |
| ALLOW document:documents:read | Read dashboards/notebooks |

### Wildcards Are NOT Supported

**Live-verified (Account Management API, 07/2026):** wildcards in permissions are **rejected** with Invalid policy statement. ALLOW or DENY must be followed by a

permission in the form `service-name:resource:action` . This applies to every position: `storage:*:read` , `storage:logs:*` , and `*:*:read` all fail.

Enumerate permissions explicitly — comma-separated lists keep statements compact:

| Instead of (invalid)                                  | Write (valid)                                                                                                                                                                                                                                  |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ALLOW</code><br><code>storage:*:read</code>     | <code>ALLOW storage:logs:read, storage:spans:read,</code><br><code>storage:metrics:read, storage:events:read,</code><br><code>storage:bizevents:read, storage:entities:read,</code><br><code>storage:system:read, storage:buckets:read;</code> |
| <code>ALLOW</code><br><code>storage:logs:*</code>     | <code>ALLOW storage:logs:read, storage:logs:write;</code>                                                                                                                                                                                      |
| <code>ALLOW</code><br><code>settings:objects:*</code> | <code>ALLOW settings:objects:read, settings:objects:write;</code>                                                                                                                                                                              |

**Also live-verified — storage WRITE permissions accept no WHERE conditions.**

`ALLOW storage:logs:write WHERE storage:bucket-name = "x"` is rejected ( Invalid condition name for permission `storage:logs:write` ). Storage writes are all-or-nothing: only `storage:logs:write` , `storage:metrics:write` , `storage:events:write` exist ( `storage:spans:write` and `storage:bizevents:write` do **not** ), and none can be scoped by `WHERE` . Scope *reads* with conditions; grant *writes* unscoped and only to trusted ingest paths — or control ingest via `OpenPipeline` instead. (Settings writes are different: `settings:objects:write` **does** accept `settings:schemaId` and `settings:dt.security_context` conditions.)

## Multiple Statements

Combine statements to build complete policies:

```
ALLOW storage:logs:read;
ALLOW storage:spans:read;
ALLOW storage:metrics:read;
ALLOW document:documents:read;
```

**Note:** Statements are additive. Multiple `ALLOW` statements combine to grant broader

access.

## 3. Services and Actions Reference

### Core Services

| Service     | Purpose                        | Common Resources                                                              |
|-------------|--------------------------------|-------------------------------------------------------------------------------|
| storage     | Grail data access              | logs , spans , metrics , events ,<br>bizevents , buckets , bucket-definitions |
| settings    | Configuration objects          | objects , schemas                                                             |
| document    | Notebooks,<br>dashboards       | documents , direct-shares , environment-<br>shares                            |
| environment | Classic roles                  | roles ( environment:roles:* )                                                 |
| state       | App state storage              | app-states , user-app-states                                                  |
| automation  | Workflows,<br>AutomationEngine | workflows , calendars , rules                                                 |

### Common Actions

| Action | Description                                                                                       |
|--------|---------------------------------------------------------------------------------------------------|
| read   | View/query data                                                                                   |
| write  | Create/modify data                                                                                |
| delete | Remove data                                                                                       |
| run    | Run (workflows: automation:workflows:run )                                                        |
| admin  | Administer all objects of a type (e.g. document:documents:admin ,<br>automation:workflows:admin ) |

## Storage Service Details

| Resource           | Description                                                                  | Actions                       |
|--------------------|------------------------------------------------------------------------------|-------------------------------|
| logs               | Log records                                                                  | read, write                   |
| spans              | Distributed traces                                                           | read                          |
| metrics            | Metric data                                                                  | read, write                   |
| events             | Platform events                                                              | read, write                   |
| bizevents          | Business events                                                              | read                          |
| entities           | Smartscape entities (Classic surface)                                        | read                          |
| system             | System/audit data (query events, internal)                                   | read                          |
| buckets            | Read records from Grail buckets (required in addition to a table permission) | read                          |
| bucket-definitions | Create and manage bucket definitions                                         | read, write, delete, truncate |

Specialized record tables follow the same `storage:<table>:read` shape — e.g.

`storage:security.events:read` , `storage:user.sessions:read` ,  
`storage:user.replays:read` , `storage:application.snapshots:read` ,  
`storage:smartscape:read` .

**Storage condition attributes:** scope `storage:*` reads with `storage:bucket-name` (e.g. `startsWith "default_"` ), `storage:table-name` (e.g. `= "logs"` ), and `storage:dt.security_context` (the Gen3 record-level scoping spine — see IAM-05).

Example — default-bucket monitoring read:

```
ALLOW storage:buckets:read WHERE storage:bucket-name startsWith "default_";
ALLOW storage:logs:read, storage:events:read, storage:metrics:read,
      storage:entities:read, storage:bizevents:read, storage:spans:read;
```

For system/audit data (the *Storage All System Data Read* managed policy), add `storage:system:read` :

```
ALLOW storage:buckets:read;
ALLOW storage:system:read;
ALLOW storage:logs:read, storage:events:read, storage:metrics:read,
      storage:entities:read, storage:bizevents:read, storage:spans:read;
```

## Settings Service Details

| Resource | Description           | Actions             |
|----------|-----------------------|---------------------|
| objects  | Configuration objects | read, write, delete |
| schemas  | Schema definitions    | read                |

## Document Service Details

| Resource           | Description                         | Actions                    |
|--------------------|-------------------------------------|----------------------------|
| documents          | Dashboards, notebooks               | read, write, delete, admin |
| direct-shares      | Direct shares with users and groups | read, write, delete        |
| environment-shares | Shares with the whole environment   | read, write, delete, claim |

Sources: [IAM policy reference \(DT docs\)](#) — permission list read 09/24/2026.

## Environment Roles ( `environment:roles:*` )

Classic environment roles are expressed as IAM permissions. This is how RBAC folds into the one policy language (see §1). All are scoped by the `environment:management-zone` attribute **except** `agent-install` and `configure-request-capture-data` (management-zone not applicable).

| Permission                                     | Classic role meaning        |
|------------------------------------------------|-----------------------------|
| <code>environment:roles:viewer</code>          | Access environment (read)   |
| <code>environment:roles:manage-settings</code> | Change monitoring settings  |
| <code>environment:roles:agent-install</code>   | Download / install OneAgent |

| Permission                                                     | Classic role meaning                |
|----------------------------------------------------------------|-------------------------------------|
| <code>environment:roles:logviewer</code>                       | View logs                           |
| <code>environment:roles:view-sensitive-request-data</code>     | View sensitive request data         |
| <code>environment:roles:configure-request-capture-data</code>  | Configure capture of sensitive data |
| <code>environment:roles:replay-sessions-with-masking</code>    | Replay session data (masked)        |
| <code>environment:roles:replay-sessions-without-masking</code> | Replay session data (unmasked)      |
| <code>environment:roles:view-security-problems</code>          | View security problems              |
| <code>environment:roles:manage-security-problems</code>        | Manage security problems            |

**Privacy / compliance note.** The session-replay pair and `view-sensitive-request-data` are the load-bearing least-privilege controls here — `replay-sessions-without-masking` and `view-sensitive-request-data` expose unmasked end-user data and should be granted narrowly (specific groups, specific management zones), never bundled into a broad editor policy.

## Extensions Service Details

Extension authorization splits into *definitions* (the extension package) and *configurations* (a deployed instance):

| Permission                                                        | Description                  | Condition attributes                                                                                                                                 |
|-------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>extensions:definitions:read</code> / <code>:write</code>    | The extension package itself | <code>extensions:extension-name</code>                                                                                                               |
| <code>extensions:configurations:read</code> / <code>:write</code> | A deployed configuration     | <code>extensions:host</code> ,<br><code>extensions:host-group</code> , <code>extensions:ag-group</code> ,<br><code>extensions:management-zone</code> |

| Permission                                                                                 | Description               | Condition attributes |
|--------------------------------------------------------------------------------------------|---------------------------|----------------------|
| <code>extensions:configuration.actions:write</code>                                        | Run configuration actions | (as above)           |
| <code>extensions:discovery.jmx:read</code> /<br><code>extensions:discovery.pmi:read</code> | Discovery surfaces        | —                    |

**Extension-ownership delegation** — let a team manage *their* extension on *their* hosts without granting tenant-wide extension rights:

```
ALLOW extensions:definitions:read, extensions:definitions:write
WHERE extensions:extension-name = "com.acme.payments";
ALLOW extensions:configurations:read, extensions:configurations:write
WHERE extensions:host-group startsWith "payments-";
```

## AppEngine Service Details ( `app-engine:*` )

| Permission                            | Description                                        |
|---------------------------------------|----------------------------------------------------|
| <code>app-engine:apps:run</code>      | See / open an app (controls navigation visibility) |
| <code>app-engine:functions:run</code> | Execute AppEngine functions                        |
| <code>app-engine:apps:install</code>  | Install apps                                       |
| <code>app-engine:apps:delete</code>   | Remove apps                                        |

App-level scoping uses the `shared:app-id` attribute (operators `=`, `!=`, `IN`, `NOT IN`, `startsWith`, `NOT startsWith`). Example — let developers install only their own apps, and grant Workflows access:

```
ALLOW app-engine:apps:install, app-engine:apps:delete WHERE shared:app-id
startsWith "my.";
ALLOW app-engine:apps:run WHERE shared:app-id = "dynatrace.automations";
```

## 4. Condition Expressions

Conditions add fine-grained control to policy statements.

**Boundary standardization (canonical pattern).** The examples in this section illustrate condition operators with literal values for clarity. In production, the literal value should be a `${bindParam:NAME}` placeholder bound at policy-binding time, with the boundary field standardized on `dt.security_context` (Gen3) or management zone (Classic). See **IAM REFERENCE.md § Policy Parameterization and Boundary Standardization** for the canonical pattern, and **IAM-10: Templated Policy-Group Assignments** for binding mechanics.

### Condition Syntax

```
ALLOW <service>:<resource>:<action> WHERE <condition>;
```

### Condition Operators

| Operator                | Description            | Example                                                             |
|-------------------------|------------------------|---------------------------------------------------------------------|
| <code>=</code>          | Equals                 | <code>storage:dt.security_context = "team-a"</code>                 |
| <code>!=</code>         | Not equals             | <code>storage:dt.security_context != "restricted"</code>            |
| <code>IN</code>         | In list                | <code>storage:dt.security_context IN ("team-a", "team-b")</code>    |
| <code>startsWith</code> | Prefix match           | <code>settings:schemaId startsWith "builtin:alerting"</code>        |
| <code>MATCH</code>      | Wildcard pattern match | <code>storage:dt.security_context MATCH( '*/app:easytrade' )</code> |

**MATCH() vs startsWith :** Use `MATCH()` for flexible wildcard patterns anywhere in the value (e.g. middle segment). Use `startsWith` when matching a fixed leading prefix. **And use `MATCH()` whenever the field can hold an array** — on an array, `=`, `startsWith` and `IN` *"always return false"*, silently. The exception is `storage:entities:read` (classic entities), which offers no `MATCH` ; there, `startsWith` is the documented operator for the multi-value field (see below). See **IAM-05** for where that bites in a boundary.

**Corrected 09/10/2026** — `contains` is not an IAM condition operator. An earlier revision of this table listed `contains ( settings:schemaId contains "custom" )`. The

IAM policy statement syntax reference gives the operator set as `= != < > IN` `startswith` `NOT IN` `NOT startswith` `MATCH` , and the IAM policy reference lists `IN` , `=` , `!=` , `startswith` and `NOT startswith` for `settings:schemaId` — `contains` appears in neither. Use `startswith` for a prefix, or `MATCH` for a pattern.

Sources: [IAM policy statement syntax and examples \(DT docs\)](#), [IAM policy reference \(DT docs\)](#), [Permissions in Grail \(DT docs\)](#) — "Using `=` , `STARTSWITH` or `IN` when the field holds an array will always return `false` ."

### Common Condition Fields

| Domain   | Field                                     | Description                      |
|----------|-------------------------------------------|----------------------------------|
| Storage  | <code>storage:dt.security_context</code>  | Security context on data         |
| Storage  | <code>storage:bucket-name</code>          | Bucket name (for team isolation) |
| Settings | <code>settings:schemaId</code>            | Settings schema                  |
| Settings | <code>settings:dt.security_context</code> | Security context on settings     |

### Security Context vs Bucket Conditions

**Default to `dt.security_context` for general team-scoped access.** Bucket conditions work well when the bucket already exists for a specific reason — compliance separation, retention isolation, or hard cost attribution. For routine team-scoped access without one of those scenarios, the `dt.security_context` condition alone is usually simpler. See [IAM REFERENCE.md § Bucket-Match Overlay](#) for the scenario gating.

| Condition Type          | Use Case                    | Example                                             |
|-------------------------|-----------------------------|-----------------------------------------------------|
| <b>Security Context</b> | Flexible, changeable access | <code>storage:dt.security_context = "team-a"</code> |
| <b>Bucket</b>           | Hard data isolation         | <code>storage:bucket-name = "team-a-logs"</code>    |
| <b>Both</b>             | Defense in depth            | Both conditions in policy + boundary                |

## Condition Examples

### Scope to security context (exact):

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "checkout"
```

### Scope to specific bucket (team isolation):

```
ALLOW storage:logs:read WHERE storage:bucket-name = "checkout_logs"
```

### Scope to settings schema:

```
ALLOW settings:objects:write WHERE settings:schemaId startsWith  
"builtin:alerting"
```

### Multiple conditions (AND):

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "team-a" AND  
storage:bucket-name = "team-a-logs"
```

### Multiple values (IN):

```
ALLOW storage:logs:read WHERE storage:dt.security_context IN ("team-a", "team-  
b", "shared")
```

### Multiple buckets (team access):

```
ALLOW storage:logs:read WHERE storage:bucket-name IN ("checkout_logs",  
"shared_logs")  
ALLOW storage:spans:read WHERE storage:bucket-name IN ("checkout_spans",  
"shared_spans")
```

### Parameterizing the boundary (recommended for production):

The examples above use literal values to illustrate operators. In production, replace each literal with a `${bindParam:NAME}` placeholder and supply the value at binding time. One policy template, N bindings — instead of N near-identical policies.

### Recommended (parameterized):

```
ALLOW storage:logs:read WHERE storage:dt.security_context =  
"${bindParam:team}";
```

Resolved when bound with `{"parameters": {"team": "checkout"}}` :

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "checkout";
```

The same approach applies to bucket-name overlays, `MATCH()` patterns, and `IN` lists. See **IAM-10: Templated Policy-Group Assignments** for binding mechanics and **IAM REFERENCE.md § Policy Parameterization and Boundary Standardization** for the full canonical pattern (including the `Gen3 = dt.security_context` standardization rule and the change-management caveat).

## Structured Security Context Design

For organisations with multiple teams and cross-cutting access requirements, a flat single-value context (e.g. `"checkout"` ) scales poorly. Instead, encode multiple dimensions into a structured key-value string:

```
comp:&lt;component&gt;/bu:&lt;business-unit&gt;/app:&lt;application&gt;
```

Examples:

| Entity                  | Security Context                               |
|-------------------------|------------------------------------------------|
| Easytrade application   | <code>comp:app/bu:digital/app:easytrade</code> |
| Easytrade database      | <code>comp:db/bu:digital/app:easytrade</code>  |
| Easytrade load balancer | <code>comp:lb/bu:digital/app:easytrade</code>  |

 **Dimension order matters for Smartscape/Classic entities.** Place the dimension you need transversal access on **first** — typically `comp` for infrastructure teams. On `storage:smartscape:read` and the record tables, use `MATCH('comp:db*')` to anchor at the start. `storage:entities:read` (**classic entities**) supports `=`, `IN` and `startswith` **only** — no `MATCH` — and Dynatrace documents `startswith` as evaluating for any matching value of the multi-value entity context.

Using `MATCH()` for flexible policy conditions:

App team — access all components within their application:

```
// 3rd Gen signals
ALLOW storage:metrics:read WHERE storage:dt.security_context
MATCH('*/app:easytrade');
ALLOW storage:logs:read WHERE storage:dt.security_context
MATCH('*/app:easytrade');
ALLOW storage:spans:read WHERE storage:dt.security_context
MATCH('*/app:easytrade');
```

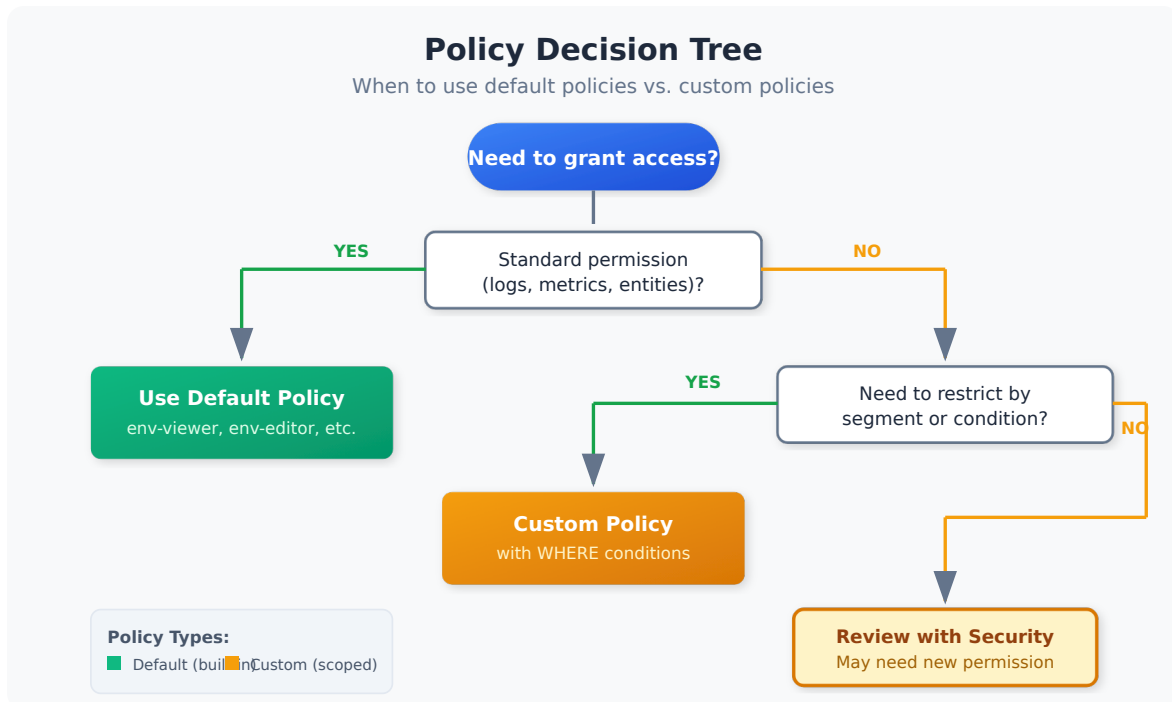
Transversal database team — access all database components across all applications:

```
// 3rd Gen signals
ALLOW storage:metrics:read WHERE storage:dt.security_context
MATCH('comp:db*');
ALLOW storage:logs:read WHERE storage:dt.security_context
MATCH('comp:db*');
ALLOW storage:spans:read WHERE storage:dt.security_context
MATCH('comp:db*');
// Classic entities — storage:entities:read has no MATCH; startsWith matches
any value of the array
ALLOW storage:entities:read WHERE storage:dt.security_context startsWith
"comp:db";
ALLOW storage:smartscape:read WHERE storage:dt.security_context
MATCH('comp:db*');
```

**Note:** `storage:entities:read` (classic entities) supports `=`, `IN` and `startsWith` only — Dynatrace documents `startsWith` as evaluating for any matching value of the multi-value entity context. `storage:smartscape:read` and the record tables ( `storage:logs:read` etc.) also accept `MATCH` ; anchor at the start with a trailing `*` to mimic a prefix match (e.g. `MATCH('comp:db*')` ). See **IAM-05** for the corresponding boundary patterns.

**Sources:** [IAM policy reference \(DT docs\)](#), [Grant access to entities with security context \(DT docs\)](#) —  
*"the security context of this entity. Can be a multi-value field and startsWith will evaluate for any matching value."*

## 5. Common Policy Patterns



### Pattern 1: Read-Only Access

```
// Read all data, no write access (wildcards are not supported – enumerate)
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      storage:events:read, storage:bizevents:read, storage:entities:read,
      storage:system:read, storage:buckets:read;
ALLOW document:documents:read;
ALLOW state:app-states:read, state:user-app-states:read;
ALLOW settings:objects:read, settings:schemas:read;
```

### Pattern 2: Team-Scoped Access (Security Context)

**Recommended (parameterized):** One policy bound per team via binding parameters. Boundary field standardized on `dt.security_context` (Gen3 universal scoping field). See **IAM REFERENCE.md § Policy Parameterization and Boundary Standardization**.

```
// Parameterized – one policy template, bound per team via binding parameters.
// Reads are condition-scoped; storage writes cannot take WHERE conditions
// (live-verified) – grant them separately and unscoped only where justified.
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
    storage:events:read, storage:bizevents:read
    WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW settings:objects:read, settings:objects:write
    WHERE settings:dt.security_context = "${bindParam:team}";
```

Resolved when bound with `{"parameters": {"team": "checkout-team"}} :`

```
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
    storage:events:read, storage:bizevents:read
    WHERE storage:dt.security_context = "checkout-team";
ALLOW settings:objects:read, settings:objects:write
    WHERE settings:dt.security_context = "checkout-team";
```

Bind the same template to each team with a different `team` value rather than duplicating the policy. See **IAM-10** for binding mechanics.

### Pattern 3: Bucket-Based Team Isolation (Recommended for Strict Isolation)

**Use this pattern** when teams must have physically separated data with no cross-team visibility.

**When this pattern fits:** bucket boundaries are well-suited to scenarios where the bucket already exists for a specific reason — compliance separation (PCI/HIPAA), retention isolation, or hard cost attribution. For general team-scoped access without one of those reasons, **Pattern 2 ( `dt.security_context` ) is usually the simpler choice**. When the bucket-based scenario does apply, parameterize per-team bucket names via binding parameters (shared bucket statements stay as literals — "shared" is a singleton, not a per-team scope). See **IAM REFERENCE.md § Bucket-Match Overlay** for scenario gating, and **§ Policy Parameterization and Boundary Standardization** for the full pattern.

```
// Parameterized — per-team data permissions (one policy template, bound per team)
ALLOW storage:logs:read    WHERE storage:bucket-name = "${bindParam:log-bucket}";
ALLOW storage:spans:read   WHERE storage:bucket-name = "${bindParam:span-bucket}";
ALLOW storage:metrics:read WHERE storage:bucket-name = "${bindParam:metric-bucket}";

// Shared buckets — literal (Class C: shared is a singleton, not per-team)
ALLOW storage:logs:read   WHERE storage:bucket-name = "shared_logs";
ALLOW storage:spans:read  WHERE storage:bucket-name = "shared_spans";
```

**Writes cannot be bucket-scoped (live-verified).** `storage:logs:write` WHERE `storage:bucket-name = ...` is rejected by the API, and `storage:spans:write` does not exist at all — spans are ingest-only. If a team needs log/metric/event write access, grant it unscoped in a separate, deliberately-assigned policy, and control WHERE data lands via OpenPipeline routing — not via IAM conditions.

**Resolved when bound with** `{"parameters": {"log-bucket": "checkout_logs", "span-bucket": "checkout_spans", "metric-bucket": "checkout_metrics"}} :`

```
ALLOW storage:logs:read    WHERE storage:bucket-name = "checkout_logs";
ALLOW storage:spans:read   WHERE storage:bucket-name = "checkout_spans";
ALLOW storage:metrics:read WHERE storage:bucket-name = "checkout_metrics";
ALLOW storage:logs:read   WHERE storage:bucket-name = "shared_logs";
ALLOW storage:spans:read  WHERE storage:bucket-name = "shared_spans";
```

### Bucket + Security Context (Defense in Depth):

**Recommended (parameterized):** Both fields parameterized; the bucket-match clause is the Gen3 data overlay on the `dt.security_context` spine.

```
// Parameterized defense-in-depth
ALLOW storage:logs:read WHERE storage:bucket-name = "${bindParam:log-bucket}"
    AND storage:dt.security_context = "${bindParam:team}";
```

**Resolved when bound with** `{"parameters": {"log-bucket": "checkout_logs", "team": "checkout"}} :`

```
ALLOW storage:logs:read WHERE storage:bucket-name = "checkout_logs"
AND storage:dt.security_context = "checkout";
```

## Pattern 4: Log Management

```
// Read all logs, manage log pipelines
ALLOW storage:logs:read;
ALLOW settings:objects:read, settings:objects:write
WHERE settings:schemaId startsWith "builtin:logmonitoring";
```

## Pattern 5: Dashboard Creator

```
// Create and share dashboards (documents:share does not exist – sharing is
direct-shares)
ALLOW document:documents:read, document:documents:write,
document:documents:delete;
ALLOW document:direct-shares:read, document:direct-shares:write;
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
storage:events:read, storage:bizevents:read;
```

## Pattern 6: Alerting Manager

```
// Manage alerting configuration
ALLOW settings:objects:read, settings:objects:write
WHERE settings:schemaId startsWith "builtin:alerting";
ALLOW settings:objects:read, settings:objects:write
WHERE settings:schemaId startsWith "builtin:problem";
ALLOW automation:workflows:read, automation:workflows:write,
automation:workflows:run;
```

## Pattern 7: SRE On-Call Access

```
// Broad read access + problem/event visibility
// (state:problems does not exist – Davis problems are events in Grail)
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
storage:events:read, storage:bizevents:read, storage:entities:read;
ALLOW document:documents:read;
ALLOW settings:objects:read;
```

## Pattern 8: Security Auditor

```
// Read everything, write nothing (enumerated – wildcards are not supported)
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      storage:events:read, storage:bizevents:read, storage:entities:read,
      storage:system:read, storage:buckets:read;
ALLOW settings:objects:read, settings:schemas:read;
ALLOW document:documents:read, document:direct-shares:read,
document:environment-shares:read;
ALLOW state:app-states:read, state:user-app-states:read;
```

## Pattern 9: Compliance-Scoped Access (Bucket-Based)

```
// Access only to PCI-compliant data bucket
ALLOW storage:logs:read WHERE storage:bucket-name = "pci_logs";
ALLOW storage:spans:read WHERE storage:bucket-name = "pci_spans";
// No access to non-PCI buckets
```

## Pattern 10: Tiered Environment Access (Management Zones)

Grant full access in lower environments and read-only in production — using `environment:roles:*` permissions scoped by the `environment:management-zone` attribute (the classic-role surface, complementary to the Grail patterns above):

```
// Full access in dev and hardening
ALLOW environment:roles:viewer, environment:roles:manage-settings
WHERE environment:management-zone IN ("dev", "hardening");

// Read-only in production
ALLOW environment:roles:viewer
WHERE environment:management-zone IN ("prod");
```

This is the canonical app-developer shape: iterate freely in non-prod, observe-only in prod. Add `environment:roles:logviewer` to the prod statement if the team needs production log access without settings rights.

## Choosing Between Patterns

| Requirement                            | Recommended Pattern                  |
|----------------------------------------|--------------------------------------|
| Teams can share data flexibly          | Pattern 2 (Security Context)         |
| Teams must never see each other's data | <b>Pattern 3 (Buckets)</b>           |
| Compliance/regulatory data separation  | <b>Pattern 9 (Bucket-Based)</b>      |
| Temporary project access               | Pattern 2 (Security Context)         |
| Cost allocation by team                | <b>Pattern 3 (Buckets)</b>           |
| Full in non-prod, read-only in prod    | <b>Pattern 10 (Management Zones)</b> |

## 6. Default vs Custom Policies

### Built-in Default Policies

Dynatrace provides default policies for common use cases:

| Policy                                                    | Description                                        | Use Case              |
|-----------------------------------------------------------|----------------------------------------------------|-----------------------|
| <b>Standard User</b>                                      | Access the environment and run Dynatrace apps      | Most users            |
| <b>Pro User</b>                                           | Standard, plus build and run apps and workflows    | Team members          |
| <b>Admin User</b>                                         | Administrative access across all platform services | Administrators        |
| <code>account-viewer</code> (account permission)          | View account                                       | Cross-team visibility |
| <code>account-user-management</code> (account permission) | Manage users, groups, policies                     | IAM administrators    |

### Reach for Managed Policies First

**Default to the policies Dynatrace ships before authoring custom ones.** Most standard scenarios — monitoring read access, app users, automation access — are already

covered by managed policies; hand-rolling a custom policy for these is extra surface to maintain and audit. Author custom policies only when no managed policy fits (team-scoped data, specific single-capability grants, compliance constraints).

Commonly used managed policies for Grail and AppEngine onboarding:

| Managed policy                        | Grants                                                                                                        |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------|
| All Grail data read access            | Unrestricted Grail read                                                                                       |
| Read <table>                          | Per-table read (e.g. <i>Read Logs</i> , <i>Read Metrics</i> , <i>Read Spans</i> , <i>Read System Events</i> ) |
| Standard User / Pro User / Admin User | App use, app and workflow building, administration (see the table above)                                      |

**Legacy default policies.** *AppEngine - User*, *AppEngine - Admin*, *AppEngine - Developer access*, *AutomationEngine - User access*, *AutomationEngine - Admin access*, *Storage All System Data Read* and *Storage Default Monitoring Read* are now listed as legacy: Dynatrace states they “are not accessible for new policy assignments, but existing assignments of these policies remain until removed.” Use the role policies and the *Read <table>* policies for new bindings, and verify the current catalog in your tenant's Account Management — it evolves.

Sources: [Default policies \(DT docs\)](#).

## When to Use Default Policies

| Scenario                 | Recommendation |
|--------------------------|----------------|
| Simple permission levels | Use defaults   |
| Quick onboarding         | Use defaults   |
| Standard roles           | Use defaults   |

## When to Create Custom Policies

| Scenario                | Recommendation |
|-------------------------|----------------|
| Team-scoped data access | Custom policy  |

| Scenario                        | Recommendation |
|---------------------------------|----------------|
| Specific capability (logs only) | Custom policy  |
| Compliance requirements         | Custom policy  |
| Least privilege needs           | Custom policy  |

## Combining Default and Custom

Groups can have both:

```
Group: dt-checkout-editors
├─ Default Policy: Standard User (base access)
└─ Custom Policy: checkout-write-access (team-specific writes)
```

This provides base read access plus targeted write permissions.

## 7. Policy Testing and Validation

### Testing Methodology

1. **Create policy in test environment** (or account)
2. **Assign to test group** with a test user
3. **Verify intended access** works
4. **Verify unintended access** is blocked
5. **Document results** and adjust

### Common Validation Tests

| Test                         | Expected Result |
|------------------------------|-----------------|
| Query logs in scope          | Success         |
| Query logs out of scope      | Empty or denied |
| Modify settings in scope     | Success         |
| Modify settings out of scope | Denied          |

| Test             | Expected Result      |
|------------------|----------------------|
| Create dashboard | Success (if allowed) |

## Debugging Policy Issues

If access isn't working as expected:

1. **Check group membership** - Is user in the correct group?
2. **Check policy assignment** - Is policy assigned to group?
3. **Check boundary** - Does boundary allow access to the entity?
4. **Check conditions** - Do condition values match?
5. **Check security context** - Is data tagged correctly?

See **IAM-09: Troubleshooting Access Issues** for detailed debugging.

## 8. GitOps for Policies

Version control your policies for audit trail and peer review.

### Policy as Code Structure

```
iam-config/
├─ policies/
│   ├─ team-checkout-access.yaml
│   ├─ team-payments-access.yaml
│   ├─ sre-oncall.yaml
│   └─ security-auditor.yaml
├─ groups/
│   └─ group-assignments.yaml
└─ README.md
```

### Policy YAML Format (Monaco)

Monaco manages IAM as **account resources** — `policies:`, `groups:` and `boundaries:` are dedicated top-level types in YAML, not Settings schemas, and no JSON templates are needed. `monaco deploy` ignores account configuration — Monaco has dedicated commands for account resources, which authenticate with an OAuth client configured under `accounts:` in `manifest.yaml`.

## Policy and group (documented account format):

```
# account/iam.yaml
policies:
  - name: team-data-reader
    id: team-data-reader
    level:
      type: account
    description: Read logs and spans — scoped per team by the boundary on each
binding
    policy: |-
      ALLOW storage:logs:read;
      ALLOW storage:spans:read;

boundaries:
  - id: bnd-team-checkout
    name: bnd-team-checkout
    query: storage:dt.security_context IN ("checkout");

groups:
  - name: dt-checkout-team
    id: dt-checkout-team
    description: Checkout team
    environments:
      - environment: abc12345
        policies:
          - policy:
              type: reference
              id: team-data-reader
            boundaries:
              - type: reference
                id: bnd-team-checkout
```

**Binding parameters are not in Monaco's documented account format.** The page documents policies, groups, boundaries and the policy-to-group binding (with boundaries), but not binding parameters, so a `${bindParam:team}` template cannot be given its value here. Bind templated policies with parameters through the IAM API or Terraform ( `dynatrace_iam_policy_bindings_v2` ) — see **IAM-10** and **IAM-95**. The example above keeps the policy unparameterized and scopes it with a per-team boundary instead.

**Sources:** [Monaco account configuration \(DT docs\)](#) — "Using Monaco, you can define users, service users, groups, policies, and boundaries as dedicated types in YAML configuration files."

## GitOps Workflow

1. **Branch:** Create feature branch for policy change
2. **Edit:** Modify policy YAML
3. **Review:** Submit PR, get peer review
4. **Test:** Deploy to test environment
5. **Approve:** Merge after validation
6. **Deploy:** CI/CD applies to production

## Benefits

| Benefit       | Description                              |
|---------------|------------------------------------------|
| Audit Trail   | Git history shows who changed what, when |
| Peer Review   | Changes reviewed before apply            |
| Rollback      | Easy revert to previous state            |
| Documentation | Policies are self-documenting            |
| Consistency   | Same process for all changes             |

See **AUTOM-03: Monaco** for full implementation details.

## 9. Policy Assessment Queries

Use these queries to audit policy-related activity.

**Group, policy, boundary and binding changes are account-level, and this environment's audit events are not their change log.** *"Dynatrace provides audit logs of all changes to your account-level identity and access (IAM) management settings" — "Administrators can view these logs in Account Management > Settings > Audit log", where they are kept for up to ten years (also queryable through the Account Audits API). The query below shows IAM API calls made **through this environment** — useful for spotting automation, not a change log. On the validation tenant, 30 days of these events held no group or policy change at all: only user lookups and scheduled `/lookups/iam_*` file uploads, which the query now excludes.*

Sources: [Account Management audit logs \(DT docs\)](#).

```
// IAM API activity in this environment (policy changes themselves: account
audit log)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

```
// Find access denied events
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter event.outcome == "403"
| fields timestamp, user.id, event.type, resource, origin.address
| sort timestamp desc
| limit 50
```

```
// Summarize permission-related audit events
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| summarize events = count(), by:{event.outcome, event.type}
| sort events desc
| limit 25
```

## Next Steps

---

With policies defined, complete your access control setup:

### Recommended Path

1. **IAM-05: Boundary Design Patterns** - Control what entities users can see
2. **IAM-06: User Lifecycle and Provisioning** - Automate user management
3. **IAM-07: Audit Logging and Compliance** - Monitor policy effectiveness
4. **IAM-10: Templated Policy-Group Assignments** - Reusable parameterized policies for team/environment scoping at scale
5. **IAM-11: Policy Persona Workshop** - Design persona-based policies from stakeholder requirements using a structured 6-goal methodology

## Policy Checklist

Before moving on, ensure you have:

- [ ] Understood policy statement syntax
  - [ ] Reviewed services and actions reference
  - [ ] Designed policies for your team structure
  - [ ] Decided on default vs custom policy usage
  - [ ] Established testing methodology
  - [ ] Considered GitOps for policy management
- 

## Summary

---

In this notebook, you learned:

- Policy fundamentals and scope levels
  - Statement syntax: `ALLOW <service>:<resource>:<action>`
  - Services and actions reference
  - Condition expressions for fine-grained control
  - Seven common policy patterns
  - When to use default vs custom policies
  - Policy testing and validation approaches
  - GitOps for policy version control
- 

## References

---

- [Policy Statements Reference](#)
- [Permission Management](#)
- [Account Policies](#)

## References

- [IAM policy statements reference \(DT docs\)](#) — authoritative permission + attribute catalog
- [Tailored access management, Part 1 — one configuration for all authorization requirements \(Dynatrace News\)](#)
- [Tailored access management, Part 2 — onboard users to Grail and AppEngine \(Dynatrace News\)](#)

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*